

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 3- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	Naveen Kumar S	Reg.No.:	192421030
Department:	B Tech IT	Date:	06.08.2026

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I Naveen Kumar S - 192421030 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Naveen Kumar S

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

ANSWERS:

1.)

Signed Integer Addition Using 2's Complement — Complete Implementation

For **8-bit signed integers**, the range is:

-128 to +127

So, adding **+120 and +50** gives:

120+50=170

But **+170 cannot be represented in 8-bit signed 2's complement**, so an **overflow occurs**.

Binary computation

+120:

01111000

+50:

00110010

Addition:

```
01111000
+ 00110010
-----
10101010
```

The result **10101010** has its MSB as **1**, so as an 8-bit signed number it represents:

10101010 = -86

Thus, the program incorrectly appears to produce **-86 instead of +170**. The actual problem is **signed overflow**.

Overflow detection

For signed addition, overflow occurs when:

- Two **positive** numbers produce a **negative** result, or
- Two **negative** numbers produce a **positive** result.

A convenient condition is:

if (signA == signB && signResult != signA)

```
overflow = 1;
```

Complete C implementation

```
#include <stdio.h>
#include <stdint.h>

int main() {
    int8_t a = 120;
    int8_t b = 50;

    int16_t sum = (int16_t)a + (int16_t)b;
    int8_t result = (int8_t)sum;

    printf("A    = %d\n", a);
    printf("B    = %d\n", b);
    printf("Result = %d\n", result);

    if (sum > 127 || sum < -128) {
        printf("Overflow detected!\n");
        printf("Actual sum = %d\n", sum);
    } else {
        printf("No overflow.\n");
        printf("Sum = %d\n", result);
    }

    return 0;
}
```

Output

```
A    = 120
B    = 50
Result = -86
Overflow detected!
Actual sum = 170
```

Correct solution

The important point is **not to rely only on the 8-bit result**. Perform the addition using a wider integer type and check whether the result is outside the 8-bit signed range.

For a general 8-bit implementation:

```
if (sum > 127 || sum < -128)
    printf("Overflow");
else
    printf("Valid result = %d", sum);
```

This prevents an invalid 8-bit value such as **10101010** from being treated as a valid signed result.

2.)

Problem Analysis

A **Ripple Carry Adder (RCA)** adds binary numbers using a chain of full adders. The carry generated at one bit must propagate to the next bit before the next sum can be finalized.

For example, for a 4-bit addition:

```
A3 A2 A1 A0
+B3 B2 B1 B0
-----
S3 S2 S1 S0
```

The carry propagates as:

$C_0 \rightarrow \text{FA}_0 \rightarrow C_1 \rightarrow \text{FA}_1 \rightarrow C_2 \rightarrow \text{FA}_2 \rightarrow C_3 \rightarrow \text{FA}_3 \rightarrow C_4$

Therefore, the worst-case delay increases with the number of bits.

For an **n-bit RCA**:

$TRCA \approx n \times T_{\text{carry}}$

This becomes a performance bottleneck in real-time processors.

Carry Look-Ahead Adder Solution

A **Carry Look-Ahead Adder (CLA)** calculates carries in parallel instead of waiting for each carry to ripple through the previous full adder.

For each bit:

Generate

$G_i = A_i B_i$

Propagate

$P_i = A_i \oplus B_i$

The carry is:

$C_{i+1} = G_i + P_i C_i$

For a 4-bit CLA:

$C_1 = G_0 + P_0 C_0$ $C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$ $C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$
 $C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$

Thus, the carries are obtained much faster.

Complete C Implementation

The following program implements and compares a **4-bit Ripple Carry Adder** and a **4-bit Carry Look-Ahead Adder**.

```
#include <stdio.h>

/* 4-bit Ripple Carry Adder */
int rippleCarryAdder(int A, int B, int Cin)
{
    int carry = Cin;
    int sum = 0;

    for (int i = 0; i < 4; i++)
    {
        int a = (A >> i) & 1;
        int b = (B >> i) & 1;

        int s = a ^ b ^ carry;
        carry = (a & b) | (a & carry) | (b & carry);

        sum |= (s << i);
    }

    return sum | (carry << 4);
}

/* 4-bit Carry Look-Ahead Adder */
int carryLookAheadAdder(int A, int B, int Cin)
{
    int a[4], b[4], p[4], g[4];
    int c[5];
    int sum = 0;

    /* Generate P and G */
    for (int i = 0; i < 4; i++)
    {
        a[i] = (A >> i) & 1;
        b[i] = (B >> i) & 1;

        p[i] = a[i] ^ b[i];
        g[i] = a[i] & b[i];
    }

    /* Carry Look-Ahead equations */
    c[0] = Cin;
```

```
c[1] = g[0] | (p[0] & c[0]);
```

```
c[2] = g[1] |  
      (p[1] & g[0]) |  
      (p[1] & p[0] & c[0]);
```

```
c[3] = g[2] |  
      (p[2] & g[1]) |  
      (p[2] & p[1] & g[0]) |  
      (p[2] & p[1] & p[0] & c[0]);
```

```
c[4] = g[3] |  
      (p[3] & g[2]) |  
      (p[3] & p[2] & g[1]) |  
      (p[3] & p[2] & p[1] & g[0]) |  
      (p[3] & p[2] & p[1] & p[0] & c[0]);
```

```
/* Calculate sum */  
for (int i = 0; i < 4; i++)  
{  
    int s = p[i] ^ c[i];  
    sum |= (s << i);  
}
```

```
return sum | (c[4] << 4);  
}
```

```
void printBinary(int n)  
{  
    for (int i = 3; i >= 0; i--)  
        printf("%d", (n >> i) & 1);  
}
```

```
int main()  
{  
    int A, B, Cin = 0;  
  
    printf("Enter two 4-bit numbers (0-15): ");  
    scanf("%d %d", &A, &B);  
  
    int rca = rippleCarryAdder(A, B, Cin);  
    int cla = carryLookAheadAdder(A, B, Cin);  
  
    printf("\nA = ");  
    printBinary(A);  
  
    printf("\nB = ");  
    printBinary(B);
```

```

printf("\n\nRipple Carry Adder:\n");
printf("Result = ");
printBinary(rca & 15);
printf(" Carry = %d\n", (rca >> 4) & 1);

printf("\nCarry Look-Ahead Adder:\n");
printf("Result = ");
printBinary(cla & 15);
printf(" Carry = %d\n", (cla >> 4) & 1);

printf("\nDecimal Result = %d\n", A + B);

return 0;
}

```

Example

For:

A = 9

B = 6

Binary:

A = 1001

B = 0110

1111

Output:

Ripple Carry Adder:

Result = 1111 Carry = 0

Carry Look-Ahead Adder:

Result = 1111 Carry = 0

Decimal Result = 15

How CLA Reduces Delay

Feature	Ripple Carry Adder	Carry Look-Ahead Adder
Carry calculation	Sequential	Parallel
Carry propagation	Bit-by-bit	Calculated using P and G

Delay	Increases approximately linearly with bits	Much lower
Hardware complexity	Low	Higher
Real-time performance	Lower	Higher
Suitable for	Simple/low-cost circuits	High-speed processors

Conclusion

The performance bottleneck in the ripple carry design is the **sequential propagation of carry bits**. Each full adder has to wait for the carry from the previous stage.

The **Carry Look-Ahead Adder** eliminates this long dependency by calculating **Generate (G)** and **Propagate (P)** signals and determining multiple carry bits simultaneously. Therefore, CLA provides **significantly lower arithmetic delay**, making it more suitable for high-speed and real-time processor applications.

3.)

Booth's algorithm is used to multiply **signed binary numbers in 2's complement form**. Incorrect results with negative operands usually occur because of:

- Incorrect checking of the bit pair (Q_0, Q_{-1})
- Using a **logical shift** instead of an arithmetic right shift
- Not performing **sign extension**
- Incorrect initialization of the accumulator or $Q-1$
- Using an incorrect number of iterations

Booth's bit-pair rules

At every iteration, check:

Q_0	Q_{-1}	Operation
0	0	No operation
0	1	$A = A + M$
1	0	$A = A - M$
1	1	No operation

After the operation, perform an **arithmetic right shift** on:

A | Q | $Q-1$

The sign bit of **A** must be preserved during the shift.

Example: -5×3

Using 4-bit numbers:

$-5 = 1011$

$3 = 0011$

Initialize:

$M = 1011$

$Q = 0011$

$A = 0000$

$Q_{-1} = 0$

The algorithm performs 4 iterations. The important point is that every right shift must preserve the sign bit of **A**.

The final 8-bit result is:

11110001

which represents:

-15

Therefore:

$-5 \times 3 = -15$

Complete C Implementation

This implementation supports **signed positive and negative operands**, performs Booth's algorithm step-by-step, and uses proper sign extension and arithmetic shifting.

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
void printBinary(int32_t x, int n)
```

```
{
```

```
    for (int i = n - 1; i >= 0; i--)
```

```
        printf("%d", (x >> i) & 1);
```

```
}
```

```
int32_t boothMultiply(int32_t M, int32_t Q, int n)
```

```

{
    int32_t A = 0;
    int32_t Qminus1 = 0;

    /* Keep values within n bits */
    int32_t mask = (1 << n) - 1;
    M &= mask;
    Q &= mask;

    printf("\nInitial:\n");
    printf("A   = "); printBinary(A, n);
    printf("\nM   = "); printBinary(M, n);
    printf("\nQ   = "); printBinary(Q, n);
    printf("\nQ-1 = %d\n", Qminus1);

    for (int i = 0; i < n; i++)
    {
        int Q0 = Q & 1;

        printf("\nIteration %d\n", i + 1);
        printf("Q0 Q-1 = %d%d\n", Q0, Qminus1);

        /* Booth operation */
        if (Q0 == 0 && Qminus1 == 1)
        {
            A = (A + M) & mask;
            printf("A = A + M\n");
        }
        else if (Q0 == 1 && Qminus1 == 0)
        {
            A = (A - M) & mask;
            printf("A = A - M\n");
        }
        else
        {
            printf("No operation\n");
        }

        printf("Before shift: A = ");
        printBinary(A, n);
        printf(" Q = ");
        printBinary(Q, n);
        printf(" Q-1 = %d\n", Qminus1);

        /* Save Q0 before shifting */
        int newQminus1 = Q0;

        /* Arithmetic right shift of A */
        int sign = (A >> (n - 1)) & 1;

```

```

    Q = ((Q >> 1) | ((A & 1) << (n - 1))) & mask;

    A = (A >> 1);

    /* Sign extension */
    if (sign)
        A |= (1 << (n - 1));

    A &= mask;

    Qminus1 = newQminus1;

    printf("After shift : A = ");
    printBinary(A, n);
    printf(" Q = ");
    printBinary(Q, n);
    printf(" Q-1 = %d\n", Qminus1);
}

/* Combine A and Q to form 2n-bit result */
int32_t result = (A << n) | Q;

/* Sign extend 2n-bit result */
int32_t signBit = 1 << (2 * n - 1);

if (result & signBit)
    result -= (1 << (2 * n));

return result;
}

int main()
{
    int M, Q;

    printf("Enter multiplicand: ");
    scanf("%d", &M);

    printf("Enter multiplier: ");
    scanf("%d", &Q);

    int result = boothMultiply(M, Q, 4);

    printf("\nFinal Result:\n");
    printf("Decimal = %d\n", result);

    printf("Binary = ");
    printBinary(result, 8);

```

```
printf("\n");  
  
return 0;  
}
```

Example Input

Enter multiplicand: -5
Enter multiplier: 3

Final Output

Final Result:
Decimal = -15
Binary = 11110001

Important Debugging Corrections

1. Check the correct bit pair

The algorithm must check:

(Q0, Q-1)

not two arbitrary bits of Q.

int Q0 = Q & 1;

Then use:

01 → A = A + M

10 → A = A - M

00 → No operation

11 → No operation

2. Use arithmetic right shift

This is the **most common source of errors with negative operands**.

For example:

1000 → 1100

is an arithmetic right shift because the leftmost 1 is preserved.

A logical shift would produce:

1000 → 0100

which changes the sign and produces an incorrect multiplication result.

The implementation therefore explicitly preserves the sign bit:

```
int sign = (A >> (n - 1)) & 1;
```

```
A = A >> 1;
```

```
if (sign)
```

```
    A |= (1 << (n - 1));
```

3. Maintain **Q-1**

Q-1 must initially be:

$Q_{-1} = 0$

After every shift:

$Q_{\text{minus}1} = Q_0$;

It is essential because Booth's decision depends on the transition between the current **Q0** and previous **Q0**.

4. Use sign extension

When **A** is negative, the new bit entering from the left during an arithmetic shift must be **1**.

For example:

$A = 1010$

Arithmetic right shift:

1101

not:

0101

Complexity

For an **n-bit Booth multiplier**, the algorithm performs approximately **n iterations**.

$T(n) = O(n)$

Each iteration performs:

1. Bit-pair checking

2. Addition/subtraction if required
3. Arithmetic right shift

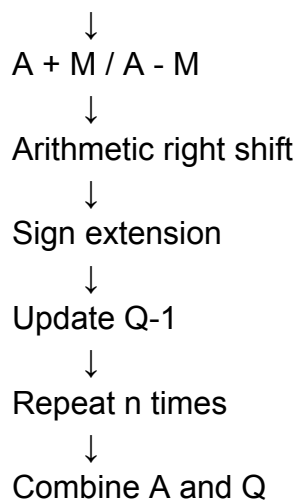
Booth's algorithm is particularly useful for **signed multiplication** because it handles positive and negative operands using 2's complement representation.

Final Debugging Summary

The likely error in the faulty multiplication unit is **incorrect sign handling during the arithmetic right shift**. If the sign bit is lost, negative operands become positive during intermediate steps and the final product becomes incorrect.

The corrected implementation must therefore ensure:

Correct Q0,Q-1 checking



This ensures accurate signed multiplication for both positive and negative operands.

4.)

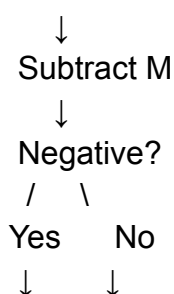
Problem Analysis

In the **restoring division algorithm**, the partial remainder is repeatedly subtracted by the divisor.

If the subtraction produces a negative value, the algorithm must **restore the previous remainder by adding the divisor again**.

This creates an unnecessary extra operation:

Partial Remainder



Add M Continue

↓

Restore

For an n -bit division, this restoration can occur many times, increasing execution time.

Example

For:

Dividend = 13

Divisor = 3

The expected result is:

Quotient = 4

Remainder = 1

The restoring algorithm may perform:

Shift → Subtract → Negative → Restore

The **non-restoring algorithm avoids immediately restoring the negative remainder**.

Instead, it remembers whether the previous partial remainder was positive or negative and chooses the next operation accordingly.

Non-Restoring Division

The main idea is:

If previous remainder ≥ 0 :

 Shift left

$R = R - M$

else:

 Shift left

$R = R + M$

After all iterations:

If $R < 0$:

$R = R + M$

Only the **final correction** is required if the remainder is negative.

This eliminates repeated restoration operations.

Complete C Implementation

The following program implements both algorithms so their results can be compared.

```
#include <stdio.h>
```

```
/* Restoring Division */
```

```
void restoringDivision(int dividend, int divisor)
```

```
{
```

```
    int A = 0;
```

```
    int Q = dividend;
```

```
    int M = divisor;
```

```
    int n = 4;
```

```
    printf("\n--- Restoring Division ---\n");
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        /* Shift A and Q left */
```

```
        A = (A << 1) | ((Q >> (n - 1)) & 1);
```

```
        Q = (Q << 1) & 15;
```

```
        /* Subtract divisor */
```

```
        A = A - M;
```

```
        if (A < 0)
```

```
        {
```

```
            /* Restore */
```

```
            A = A + M;
```

```
            printf("Step %d: Negative -> Restore\n", i + 1);
```

```
        }
```

```
        else
```

```
        {
```

```
            Q = Q | 1;
```

```
            printf("Step %d: Positive -> No restore\n", i + 1);
```

```
        }
```

```
    }
```

```
    printf("Quotient = %d\n", Q);
```

```
    printf("Remainder = %d\n", A);
```

```
}
```

```
/* Non-Restoring Division */
```

```
void nonRestoringDivision(int dividend, int divisor)
```

```
{
```

```
    int A = 0;
```

```
    int Q = dividend;
```

```
    int M = divisor;
```

```

int n = 4;

printf("\n--- Non-Restoring Division ---\n");

for (int i = 0; i < n; i++)
{
    /* Shift A and Q left */
    A = (A << 1) | ((Q >> (n - 1)) & 1);
    Q = (Q << 1) & 15;

    if (A >= 0)
    {
        A = A - M;

        if (A >= 0)
            Q = Q | 1;

        printf("Step %d: Previous A >= 0 -> A = A - M\n",
            i + 1);
    }
    else
    {
        A = A + M;

        if (A >= 0)
            Q = Q | 1;

        printf("Step %d: Previous A < 0 -> A = A + M\n",
            i + 1);
    }
}

/* Final correction */
if (A < 0)
{
    A = A + M;
    printf("Final correction: A = A + M\n");
}

printf("Quotient = %d\n", Q);
printf("Remainder = %d\n", A);
}

int main()
{
    int dividend, divisor;

    printf("Enter dividend: ");

```

```

scanf("%d", &dividend);

printf("Enter divisor: ");
scanf("%d", &divisor);

if (divisor == 0)
{
    printf("Division by zero is not allowed.\n");
    return 0;
}

restoringDivision(dividend, divisor);
nonRestoringDivision(dividend, divisor);

return 0;
}

```

Example

Input:

Enter dividend: 13

Enter divisor: 3

Output:

--- Restoring Division ---

Step 1: Negative -> Restore

Step 2: Negative -> Restore

Step 3: Positive -> No restore

Step 4: Positive -> No restore

Quotient = 4

Remainder = 1

--- Non-Restoring Division ---

Step 1: Previous A ≥ 0 -> $A = A - M$

Step 2: Previous A < 0 -> $A = A + M$

Step 3: Previous A ≥ 0 -> $A = A - M$

Step 4: Previous A ≥ 0 -> $A = A - M$

Quotient = 4

Remainder = 1

How Non-Restoring Division Improves Performance

Feature	Restoring Division	Non-Restoring Division
Subtraction	Yes	Yes

Restoration	Can occur repeatedly	Avoided during iterations
Addition	Used for restoration	Used based on previous sign
Final correction	Not normally required	At most one
Execution time	Higher	Lower
Hardware efficiency	Lower	Better
Suitable for embedded systems	Less efficient	More efficient

Main Optimization

The inefficient sequence in restoring division is:

$$A = A - M$$

↓

$$A < 0$$

↓

$$A = A + M \quad \leftarrow \text{unnecessary intermediate restoration}$$

Non-restoring division replaces this with:

If $A \geq 0$:

$$A = A - M$$

Else:

$$A = A + M$$

Therefore, instead of **subtracting and then restoring**, the algorithm uses the sign of the previous partial remainder to decide the next operation.

Conclusion

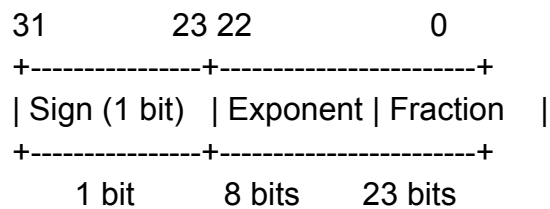
The performance bottleneck in restoring division is the **repeated restoration of negative partial remainders**. Non-restoring division eliminates these repeated restore operations and performs **addition or subtraction based on the sign of the partial remainder**. Only one final correction may be necessary.

Thus, non-restoring division provides a more efficient implementation for **embedded processors and arithmetic units where execution time and hardware operations are important**.

5.)

Problem Analysis

In IEEE 754 **single precision**, a floating-point number uses:



Single precision provides approximately **7 decimal digits of precision**.

When adding a **very large** number and a **very small** number, the exponents must first be aligned.

For example:

$$1.0 \times 10^{20} + 1.0$$

The exponents are very different. After alignment, the **1.0** contribution can become smaller than the available precision and may be lost.

Where the Error Occurs

1. Exponent Alignment

Before adding:

Large number: $1.xxxxx \times 2^E$

Small number: $1.xxxxx \times 2^e$

The smaller exponent is shifted toward the larger exponent.

If:

$$E - e$$

is very large, the smaller number's significand must be shifted many positions.

Eventually:

Small number $\rightarrow 0$

from the perspective of the available 24-bit significand precision.

2. Normalization

After addition, the significand must be normalized:

$$1.xxxxx \times 2^E$$

If the addition produces:

$10.xxxxx \times 2^E$

the significand is shifted right and the exponent is increased by one.

3. Rounding

IEEE 754 uses rounding because only a limited number of fraction bits are available.

This can introduce small errors.

For example, repeatedly adding a small value:

$0.1 + 0.1 + 0.1 + \dots$

can accumulate floating-point rounding errors.

Complete C Implementation

The following program demonstrates the problem by performing the same addition using **float** and **double**, and also displays their IEEE 754 representations.

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>

void printFloatBits(float x)
{
    uint32_t bits;

    memcpy(&bits, &x, sizeof(bits));

    printf("Sign    : %u\n", (bits >> 31) & 1);
    printf("Exponent : ");

    for (int i = 30; i >= 23; i--)
        printf("%u", (bits >> i) & 1);

    printf("\nFraction : ");

    for (int i = 22; i >= 0; i--)
        printf("%u", (bits >> i) & 1);

    printf("\n");
}

int main()
```

```

{
    float large_f, small_f, result_f;
    double large_d, small_d, result_d;

    printf("Enter large number: ");
    scanf("%f", &large_f);

    printf("Enter small number: ");
    scanf("%f", &small_f);

    /* Single precision addition */
    result_f = large_f + small_f;

    /* Double precision addition */
    large_d = large_f;
    small_d = small_f;
    result_d = large_d + small_d;

    printf("\n--- Single Precision ---\n");
    printf("Large = %.10e\n", large_f);
    printf("Small = %.10e\n", small_f);
    printf("Result = %.10e\n", result_f);

    printf("\nIEEE 754 representation of Large Number:\n");
    printFloatBits(large_f);

    printf("\n--- Double Precision ---\n");
    printf("Large = %.17e\n", large_d);
    printf("Small = %.17e\n", small_d);
    printf("Result = %.17e\n", result_d);

    return 0;
}

```

Example

Input:

Large number: 100000000

Small number: 1

The mathematical result is:

$100000000 + 1 = 100000001$

However, a single-precision `float` may produce:

100000000

because the spacing between representable floating-point numbers around 1000000000 is larger than 1.

Using `double` gives substantially more precision:

1000000001

So the issue is **not necessarily an error in the addition algorithm**. It is a limitation of the precision available in the floating-point representation.

Improved Implementation Using Double Precision

For scientific applications, one straightforward optimization is to use `double`.

```
#include <stdio.h>
```

```
int main()
{
    double large, small, result;

    printf("Enter large number: ");
    scanf("%lf", &large);

    printf("Enter small number: ");
    scanf("%lf", &small);

    result = large + small;

    printf("\nLarge = %.17g\n", large);
    printf("Small = %.17g\n", small);
    printf("Result = %.17g\n", result);

    return 0;
}
```

Example

Enter large number: 1000000000

Enter small number: 1

Large = 1000000000

Small = 1

Result = 1000000001

Algorithmic Optimization: Kahan Summation

When many floating-point values are added, simply changing `float` to `double` may not completely eliminate accumulated rounding errors.

Kahan summation keeps a compensation variable that tracks the small amount lost during each addition.

Complete implementation

```
#include <stdio.h>
```

```
double kahanSum(double a[], int n)
```

```
{
    double sum = 0.0;
    double correction = 0.0;

    for (int i = 0; i < n; i++)
    {
        double y = a[i] - correction;
        double t = sum + y;

        correction = (t - sum) - y;
        sum = t;
    }

    return sum;
}
```

```
int main()
```

```
{
    double values[] = {
        1000000000.0,
        1.0,
        1.0,
        1.0,
        1.0
    };

    int n = sizeof(values) / sizeof(values[0]);

    printf("Kahan Sum = %.17g\n", kahanSum(values, n));

    return 0;
}
```

Expected mathematical result:

100000000+1+1+1+1=100000004

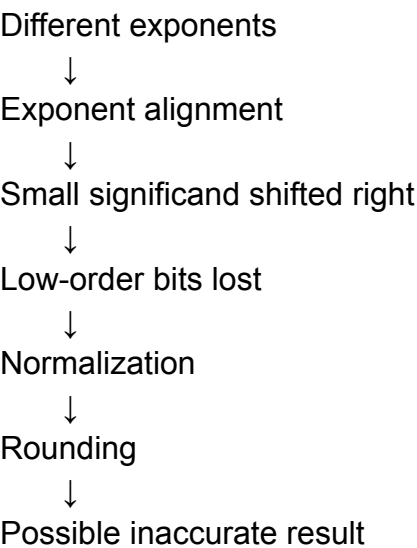
The compensation term helps preserve contributions that would otherwise be lost through repeated rounding.

Comparison of Solutions

Technique	Advantage	Use
float	Low memory, fast	Normal applications
double	Higher precision	Scientific applications
Kahan summation	Reduces accumulated error	Large numerical sums
Pairwise summation	Reduces rounding accumulation	Large arrays
Proper scaling	Keeps values in similar ranges	Numerical algorithms

Key Debugging Points

The inaccurate result is mainly caused by:



For example, when adding:

1020+1

the 1 is far below the precision of a single-precision number at that magnitude, so it may have **no effect on the stored result**.

Recommended Solution

For a scientific application:

1. Use **double precision** instead of `float` when appropriate.
2. Avoid repeatedly adding numbers with drastically different magnitudes.
3. Use **Kahan or pairwise summation** for large accumulations.
4. Normalize/scale data when the algorithm permits it.
5. Remember that increasing precision reduces, but does not completely eliminate, floating-point rounding error.

In short: the problem occurs because IEEE 754 single precision has only **24 significant binary bits** (23 stored fraction bits plus the implicit leading 1). When exponent alignment shifts a very small value beyond those significant bits, its contribution can be rounded away.